

Key-Value Store v3

Replica primary-secondary, commit e letture stantie

Architetture dei Sistemi Distribuiti

Lezione 12

Con un solo nodo bastava chiedersi:

- la write e' in RAM?
- e' su disco?

Con due nodi la domanda cambia:

che cosa significa OK quando esistono piu' copie dello stato?

Introduciamo due ruoli:

- **primary**: punto di ingresso delle scritture;
- **secondary**: destinatario della replica e possibile sorgente di letture.

La sintassi del client cambia poco. Il significato operativo della scrittura cambia molto.

Commit Locale Contro Commit Replicato

Distinzione fondamentale:

- **commit locale**: l'update e' presente sul primary;
- **commit replicato**: l'update e' presente anche sul secondary.

Se il contratto verso il client non esplicita quale dei due stia promettendo, il significato di SET resta ambiguo.

Sequenza astratta:

- `ApplyPrimary` -> `ReplyOK` -> `ReplicateLater`

Conseguenze:

- scritture piu' rapide;
- maggiore disponibilita' apparente;
- possibilita' di letture stantie dal secondario.

```
with self._lock:
    self._data[key] = value
self._replicate_async({
    "op": "SET",
    "key": key,
    "value": value,
})
return "OK", False
```

Qui OK arriva prima che il secondario abbia necessariamente visto l'update.

Trace Di Stale Read

```
Client -> primary: SET course ads  
primary -> Client: OK  
Client -> secondary: GET course  
secondary -> Client: NOT_FOUND
```

Questo non implica che il primary sia sbagliato localmente. Implica che il contratto distribuito sia debole.

Perche' La Lettura E' Diventata Ambigua

Due letture con la stessa sintassi:

- GET course dal primary;
- GET course dal secondary;

possono ora osservare stati diversi. Quindi l'API da sola non basta piu' a descrivere il comportamento. Conta anche:

- da quale replica si legge;
- in quale fase della propagazione ci si trova.

Sequenza astratta:

- `SendReplica -> WaitAck -> ApplyPrimary -> ReplyOK`

Conseguenze:

- OK piu' forte;
- maggiore latenza della scrittura;
- se il secondary non risponde, il primary non completa.

```
with self._lock:
    replicated, message = self._replicate_and_ack({
        "op": "SET",
        "key": key,
        "value": value,
    })
    if not replicated:
        return message, False
    self._data[key] = value
return "OK", False
```

Qui il secondary entra direttamente nella condizione di successo della write.

Che Cosa Migliora E Che Cosa Peggiora

Con la replica sync:

- migliora la forza semantica della risposta positiva;
- migliora l'allineamento tra primary e secondary dopo OK;
- peggiora la liveness in presenza di guasti o ritardi del secondario.

Non c'è magia: stiamo scambiando disponibilit  di scrittura con forza del commit.

Safety:

- il primary sync non promette scritture non ackate dal secondario;
- il secondario applica solo update provenienti dal primary;
- in assenza di nuovi update, i nodi convergono.

Liveness:

- async scrive anche se il secondario e' giu';
- sync si blocca sulle scritture se il secondario non collabora.

Async:

- Receive -> ApplyLocal -> Reply -> Replicate

Sync:

- Receive -> Replicate -> AckSecondary -> ApplyLocal -> Reply

La posizione della Reply nell'automa spiega da sola gran parte del cambio di contratto.

Con secondario lento e primary async:

- 1 eseguire SET course ads sul primary;
- 2 leggere subito dal secondario;
- 3 rileggere dopo il delay.

Il lag di replica diventa osservabile direttamente dal client.

Con secondario fermo:

- 1 provare una SET su primary async;
- 2 ripetere su primary sync;
- 3 confrontare disponibilita' apparente e forza semantica del commit.

Il Problema Ancora Aperto

Questa lezione non risolve ancora:

- chi decide chi e' primary;
- cosa succede se due nodi si credono entrambi primary;
- come evitare split brain.

La replica da sola non basta. Serve una nozione piu' robusta di coordinamento dei ruoli.

La stessa SET cambia di nuovo significato. Ora il contratto dipende anche da:

- quando la replica viene inviata;
- quando viene ackata;
- da quale copia arrivano le letture.

L'API resta quasi uguale. Il suo contenuto semantico continua a spostarsi con l'architettura.